

Week 13

1. Sockets (II) – Examples

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, November 24th, 2025

DEFYXHQGRUOSGRP RN	3236343972474116
QZRMVQJXVRI DKUEVP	893833235594578
KGWBGQFQHP RSUP RU	0310019701552750
XLRNYLXZDCSYU M L	7875465400196596
QZRMVQJXVRI DKUEVP	893833235594578



University of Windsor

Content

Socket based Client/Server IPC

- IP addresses and port numbers
 - Port numbers
 - IP addresses
- Examples: Implementation of client/server applications
 - Example 1: Server handles each client
 - Example 2: Server creates a child process for each client

Part I

Client/Server Typical Scenario

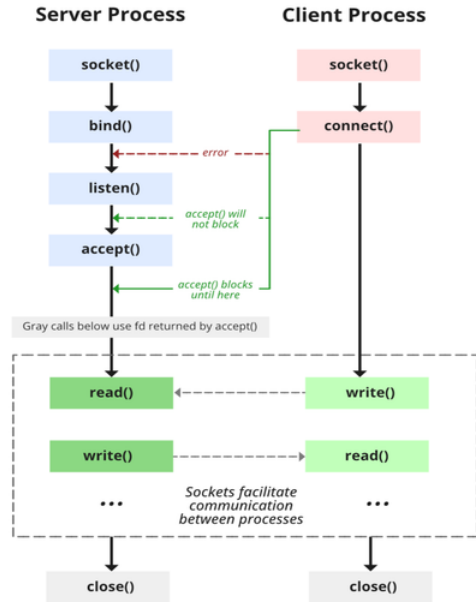


○ Typical Scenario: Client/Server



Socket Flow: Review

- Server:
 - Create Socket
 - `setsockopt`
 - Bind
 - Listen
 - Accept
 - Send/Receive
 - Close
- Client:
 - Create Socket
 - Connect
 - Send/Receive
 - Close
- Common Issues:
 - Connection failure
 - Port binding errors (already in use)
 - Blocking sockets (default)



Part II

IP Address:Port Review





Port Numbers



Port numbers

Port Numbers: Port numbers are 16-bit numbers used by TCP and UDP to identify different applications.

Three kinds of port numbers:

1. **Well known ports** are those numbered from 0 – 1023.
They are assigned by the IANA (Internet Assigned Numbers Authority), only to be used by system processes. For example:
 1. **ftp** server uses port 21,
 2. **telnet** server uses port 23, and
 3. **web** server uses port 80.Well-known port numbers are contained in file /etc/services on Unix machines.
2. **Registered Ports** are from 1024 to 49151.
They are registered by the I.A.N.A.
3. **Dynamic and/or Private Ports** are from 49152 to 65535.
Free of use.

IP Addresses



IP addresses

- Each machine on the internet has a unique Internet Address(IP address). The address is made of 32-bit number and normally written as four decimal numbers.
- E.g. arc1 : 137.207.32.27
- The command `/usr/sbin/nslookup` allows to get the IP address of a machine given its name (and vice versa).

```
127.0.0.1      localhost
137.207.82.48 ipa.cs.uwindsor.ca ipa
137.207.82.51 alpha.cs.uwindsor.ca alpha
137.207.82.52 bravo.cs.uwindsor.ca bravo
137.207.82.53 charlie.cs.uwindsor.ca charlie
```

IP addresses

```

struct in_addr {
    in_addr_t s_addr;
};

struct sockaddr_in {
    sa_family_t sin_family;
    in_port_t sin_port;
    struct in_addr sin_addr;
    unsigned char __pad[X];
};

```

/* IPv4 4-byte address */
/* Unsigned 32-bit integer */
/* IPv4 socket address */
/* Address family (AF_INET) */
/* Port number */
/* IPv4 address */
/* Pad to size of 'sockaddr' structure (16 bytes) */

Remember casting?

Cast **sockaddr_in** type to **sockaddr** type when.....?

```

struct sockaddr {
    sa_family_t sa_family;
    char sa_data[14];
};

```

/* Address family (AF_* constant) */
/* Socket address (size varies according to socket domain) */

C

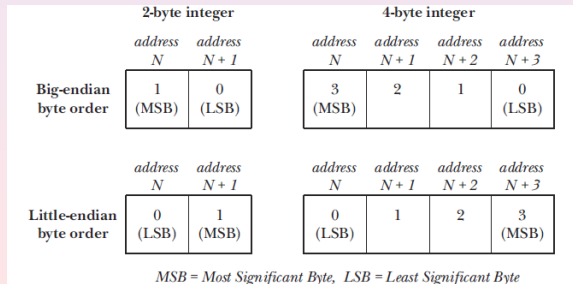
```

struct sockaddr_in server_addr;
// ... fill in server_addr details ...
bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));

```

IP addresses and Port numbers

- IP addresses and port numbers are integer values.
- One problem we encounter when passing these values across a network is that different hardware architectures store the bytes of a multibyte integer in different orders.
- The byte ordering used on a particular machine is called the **host byte order**.

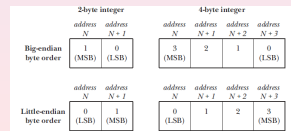


IP addresses and Port numbers

Since port numbers and IP addresses must be transmitted between, and understood by, all hosts on a network, a standard ordering must be used. This ordering is called **network byte order**, and happens to be big endian.

Operating system	Processor architecture	Byte order
FreeBSD 8.0	Intel Pentium	little-endian
Linux 3.2.0	Intel Core i5	little-endian
Mac OS X 10.6.8	Intel Core 2 Duo	little-endian
Solaris 10	Sun SPARC	big-endian

lscpu | grep "Byte Order"



MSB = Most Significant Byte, LSB = Least Significant Byte

IP addresses and Port numbers

```
#include <arpa/inet.h>
```

```
uint16_t htons(uint16_t host_uint16);
```

Returns *host_uint16* converted to network byte order

```
uint32_t htonl(uint32_t host_uint32);
```

Returns *host_uint32* converted to network byte order

```
uint16_t ntohs(uint16_t net_uint16);
```

Returns *net_uint16* converted to host byte order

```
uint32_t ntohl(uint32_t net_uint32);
```

Returns *net_uint32* converted to host byte order

IP addresses and Port numbers

C

```
uint16_t htons(uint16_t hostshort);
```

C

```
#include <arpa/inet.h>
#include <stdio.h>

int main() {
    uint16_t host_port = 8080;
    uint16_t network_port;

    network_port = htons(host_port);

    printf("Host port: %u\n", host_port);
    printf("Network port: %u\n", network_port);

    return 0;
}
```

Part III

Examples



○ Example 1: `server.c/client.c`



Example 1: Server

```
int main(int argc, char *argv[]){
    char buffer[100] = "Hello, here is my message\n";

    int sd, client;
    socklen_t len;

    struct sockaddr_in servAddr; //server socket address
    struct sockaddr_in cliAddr; //client socket address

    sd = socket(AF_INET, SOCK_STREAM, 0);

    servAddr.sin_family = AF_INET;
    servAddr.sin_addr.s_addr = inet_addr("10.60.8.51");
    // or use INADDR_ANY;
    // INADDR_ANY allows your program to work without knowing the IP address of the machine it was running on

    servAddr.sin_port = 17777; // a port number

    // The following two statements were added in case port is not released by the kernel in a timely fashion
    // when running the client many times.
    /* int yes=1;
       if (setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int)) == -1) {
           perror("setsockopt");
           exit(1);
       }
    */

    //all steps above are for calling bind.
    int rt;
    rt=bind(sd, (struct sockaddr*)&servAddr, sizeof(servAddr)); //rt for gdb to view returned value

    rt=listen(sd, 5);

    while(1){
        len = sizeof(cliAddr);
        client=accept(sd, (struct sockaddr*)&cliAddr, &len);
        //accept (sd, NULL, NULL);
        rt=write(client, buffer, strlen(buffer) + 1);
        close(client);
        break;
    }
}
```

server.c/client.c

Example 1: Client

```
int main(int argc, char *argv[]){

    char buffer[100];
    int csd;

    socklen_t len;
    struct sockaddr_in servAdd;//server socket address

    csd = socket(AF_INET, SOCK_STREAM, 0);

    servAdd.sin_family = AF_INET;
    servAdd.sin_addr.s_addr = inet_addr(argv[1]); //10.68.8.51
    // The inet_addr() function converts the specified string, in the Internet standard dot notation,
    // to an integer value suitable for use as an Internet Address.

    servAdd.sin_port = 17777; //change it to sth else?

    int rt=connect(csd, (struct sockaddr *) &servAdd, sizeof(servAdd));
    if (rt==-1)
        perror("connect() failed:");
    read(csd, buffer, 100);

    fprintf(stderr, "%s\n", buffer);
}
```

○ Example 2: `server.c/client.c` (parent/child)



Example 2: Server (Parent process)

server2.c/client2.c

```
void child(int sd);

int main(int argc, char *argv[]){
    char buffer[100];
    int sd, cd;
    socklen_t len;

    struct sockaddr_in servAdd, cliAdd;

    sd = socket(AF_INET, SOCK_STREAM, 0);

    servAdd.sin_family = AF_INET;
    servAdd.sin_addr.s_addr = INADDR_ANY;
    servAdd.sin_port = 8777;

    bind(sd, (struct sockaddr*)&servAdd, sizeof(servAdd));

    listen(sd, 5);

    while(1){
        len = sizeof(cliAdd);
        cd = accept(sd, (struct sockaddr *) &cliAdd, &len);

        //in case you want to know the client's IP address.
        char *cli_addr= inet_ntoa(cliAdd.sin_addr);
        fprintf(stderr, "Client connected from %s\n", cli_addr);

        if(fork()==0)
            child(cd);

        close(cd);
    }
}
```

Example 2: Server (Child process)

server2.c/client2.c

```
void child(int cd){
    char line[255];

    while(1){
        fprintf(stderr, "Enter a line to send to client\n");
        scanf("%s", line);      //scanf is not the best choice here (why?), try fgets (...)
        //fgets(line, 255, stdin);
        //fprintf(stderr, "line read: %s\n", line);

        write(cd, line, strlen(line)+1);
        if(!read(cd, line, 255)){
            close(cd);
            exit(0);
        }
        fprintf(stderr, "Client sent back: %s\n", line);
    }
}
```

Example 2: Client

server2.c/client2.c

```
int main(int argc, char *argv[]){
    char buffer[100];
    int csd;  socklen_t len;
    struct sockaddr_in servAdd; //server socket address

    csd = socket(AF_INET, SOCK_STREAM, 0);

    servAdd.sin_family = AF_INET;
    servAdd.sin_addr.s_addr = inet_addr(argv[1]);
    servAdd.sin_port = 8777;

    int rt = connect(csd, (struct sockaddr *) &servAdd, sizeof(servAdd));
    //fprintf(stderr, "rt=%d \n", rt);

    while(1){
        read(csd, buffer, 100);
        fprintf(stderr, "Server's message: %s\n", buffer);
        fprintf(stderr, "Enter a line to send server\n");

        scanf("%s", buffer);          //scanf is not the best choice here (why?), try fgets (...)
        //fgets(buffer, 100, stdin);

        if(buffer[0]=='$'){
            close(csd);
            exit(0);
        }
        write(csd, buffer, strlen(buffer)+1);
    }
}
```

Example 3: `server3.c` / `client3.c`

Example 4: `server4.c` / `client4.c`

Example 5: `timeserv.c` / `timeclient.c`

Example 6: `rlsd.c` / `rls.c`

← Homework

Part IV

Client/Server Socket Programming: Review



Server/client socket programming: Summary

`socklib.c`

(`make_server_socket` and `connect_to_server`
functions)

Server/client socket programming: Summary

Server normally looks like:

```
main()
{
    int sock, fd;          /* socket and connection */

    sock = make_server_socket( port );
    if ( sock == -1 ) exit(1);

    while(1){
        fd = accept( sock, NULL, NULL );
        if ( fd == -1 ) break;          /* err      */
        process_request( fd );          /* chat     */
        close(fd);                      /* hangup   */
    }
}
```

Server/client socket programming: Summary

Client normally looks like:

```
main()
{
    int fd;

    fd = connect_to_server( host, port );
    if ( fd == -1 ) exit(1);
    talk_with_server( fd );
    close(fd);
}
```

The time_server program could look like:

```
talk_with_server(fd)
{
    char buf[LEN];
    int n;

    n=read(fd, buf, LEN);
    write(1, buf, n);
}
```

```
process_request(fd)
{
    time_t now;
    char    *cp;

    time( &now );
    cp = ctime( &now );
    write(fd, cp, strlen(cp));
}
```

Another way to do it:

wait or not wait?

```
process_request (fd)
{
    int  pid = fork();

    if ( pid == -1 ) return;

    if ( pid != 0 ){
        wait(NULL);
        return;
    }
    dup2(fd, 1);
    close(fd);
    execl("date", "date", NULL);
    exit(1);
}
```

Part V
Attendance Quiz (Not graded)



Quiz



Attendance Quiz

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL QDZC
NYDWIGL OUPJL QDZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111111100
00000111011011
11001100111100

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor